Week 5: Sensors & ADC Applications - Detailed Notes

Table of contents

1 Introduction

This week covers the fundamentals of interfacing analog sensors with microcontrollers. Real embedded systems must interact with the physical world, and sensors are the bridge between physical quantities (temperature, light, pressure, etc.) and the digital domain where our firmware operates.

We will cover:

- Sensor types and their electrical characteristics
- Analog-to-Digital Converter (ADC) operation and configuration
- Signal conditioning techniques
- Digital filtering algorithms
- Calibration methods for accurate measurements

2 Sensor Fundamentals

2.1 What is a Sensor?

A sensor is a transducer that converts one form of energy into another. In embedded systems, we typically convert physical quantities into electrical signals that can be measured by the microcontroller.

Common sensor classifications:

Physical Quantity	Sensor Examples	Output Type
Temperature	LM35, NTC thermistor, RTD	Voltage, Resistance
Light	LDR, photodiode, phototransistor	Resistance, Current
Pressure	Piezoelectric, strain gauge	Voltage, Resistance
Distance	Ultrasonic, IR, LIDAR	Pulse width, Digital
Acceleration	MEMS accelerometer	Voltage, I2C/SPI
Magnetic field	Hall effect sensor	Voltage

2.2 Sensor Output Types

2.2.1 Resistive Sensors

Resistive sensors change their resistance based on the measured quantity. Examples include:

- **Thermistors (NTC/PTC):** Resistance varies with temperature
- **LDR (Light Dependent Resistor):** Resistance varies with light intensity
- **Strain gauges:** Resistance varies with mechanical strain

To interface resistive sensors with an ADC, we need a voltage divider circuit:

$$V_{out} = V_{CC} \times \frac{R_{sensor}}{R_{fixed} + R_{sensor}}$$

Design considerations:

1. Choose R_{fixed} to match the midpoint of the sensor's operating range
2. This maximizes sensitivity in the region of interest
3. Consider the current draw and power dissipation

2.2.2 Voltage Output Sensors

Voltage output sensors produce a voltage proportional to the measured quantity. Examples include:

- **LM35:** Outputs 10mV per degree Celsius
- **Potentiometers:** Voltage divider with mechanical adjustment
- **Analog accelerometers:** Output voltage proportional to acceleration

These can be connected directly to the ADC input (assuming the voltage range is appropriate).

2.2.3 Current Output Sensors

Industrial sensors often use 4-20mA current loops for noise immunity. To interface these with a voltage-input ADC, use a precision shunt resistor:

$$V_{out} = I_{sensor} \times R_{shunt}$$

For a 4-20mA sensor with 165Ω shunt: $V_{out} = 0.66V$ to $3.3V$

2.3 Transfer Functions

The transfer function describes the relationship between the input (physical quantity) and output (electrical signal).

Linear sensors:

$$Output = m \times Input + b$$

Where m is the sensitivity (slope) and b is the offset.

Non-linear sensors (e.g., thermistors):

The Steinhart-Hart equation for NTC thermistors:

$$\frac{1}{T} = A + B\ln(R) + C(\ln(R))^3$$

Or the simplified Beta equation:

$$R = R_0 \exp\left[\beta\left(\frac{1}{T} - \frac{1}{T_0}\right)\right]$$

Where:

- R_0 is resistance at reference temperature T_0 (usually 25°C = 298.15K)
- β is the material constant (typically 3000-5000K)
- T is absolute temperature in Kelvin

3 ADC Architecture

3.1 Successive Approximation Register (SAR) ADC

The STM32F401RE uses a 12-bit SAR ADC. Understanding how it works helps in configuring it correctly.

Operation principle:

1. Sample phase: Input voltage is captured on a sample-and-hold capacitor
2. Conversion phase: Binary search algorithm determines the digital value

Binary search process (12 iterations):

```
Iteration 1: Test bit 11 (MSB) → Compare with input → Keep or clear
Iteration 2: Test bit 10 → Compare → Keep or clear
...
Iteration 12: Test bit 0 (LSB) → Compare → Keep or clear
```

Conversion time:

$$t_{conv} = t_{sample} + 12 \times t_{ADC\_CLK}$$

3.2 Resolution and Quantization

12-bit resolution provides:

- 4096 discrete levels (0 to 4095)
- LSB (Least Significant Bit) = $V_{ref}/4096$
- With 3.3V reference: LSB = 0.806mV

Quantization error:

- Maximum error: ± 0.5 LSB = ± 0.4mV
- This is the inherent error from discretizing a continuous signal

Effective Number of Bits (ENOB):

In practice, noise reduces the effective resolution:

$$ENOB = \frac{SINAD - 1.76}{6.02}$$

Typical STM32 ADC ENOB: 10-11 bits (noise limits practical resolution)

3.3 Reference Voltage

The reference voltage determines the full-scale range:

$$V_{in} = \frac{ADC_{raw}}{2^{12} - 1} \times V_{ref}$$

STM32F401RE options:

Source	Voltage	Accuracy	Use Case
VDDA	~3.3V	±5%	General purpose
External VREF+	User-defined	Depends on source	Precision measurements
Internal 1.21V	1.21V ±3%	Moderate	Ratiometric

Important: VDDA variation directly affects measurement accuracy. For precise measurements, measure VDDA using the internal reference channel.

3.4 Sampling Time Considerations

Why sampling time matters:

The internal sample-and-hold capacitor must fully charge to the input voltage. Required charging time depends on:

1. Source impedance
2. Internal resistance (~6kΩ for STM32)
3. Sample capacitor (~4pF)

RC time constant:

$$\tau = (R_{source} + R_{internal}) \times C_{sample}$$

Rule of thumb: Allow 5-7 time constants for settling to within 1 LSB.

STM32 sampling time options:

Cycles	Time @ 18MHz ADC clock	Max source impedance
3	0.17µs	~1kΩ
15	0.83µs	~10kΩ
84	4.67µs	~50kΩ
480	26.67µs	~100kΩ

4 Signal Conditioning

4.1 Voltage Dividers for Resistive Sensors

Design procedure:

1. Determine sensor resistance range (R_min to R_max)
2. Choose R_fixed to center the output voltage range
3. Calculate output voltage range
4. Verify current consumption is acceptable

Example: LDR Interface

LDR specifications:

- Bright light: R = 1kΩ
- Room light: R = 10kΩ
- Dark: R = 100kΩ

With R_fixed = 10kΩ (top position):

Condition	R_LDR	V_out
Bright	1kΩ	0.30V
Room	10kΩ	1.65V
Dark	100kΩ	3.00V

Output spans 0.3V to 3.0V - good use of ADC range.

4.2 Anti-Aliasing Filters

Nyquist theorem: Sample rate must be $> 2\times$ signal bandwidth to avoid aliasing.

RC low-pass filter:

$$f_c = \frac{1}{2\pi RC}$$

Design example:

For 10 Hz signal bandwidth with 100 Hz sample rate:

- Choose $f_c = 50$ Hz (between signal and Nyquist)
- With C = 100nF: R = 31.8kΩ → use 33kΩ

Filter attenuation:

$$|H(f)| = \frac{1}{\sqrt{1 + (f/f_c)^2}}$$

At $f = 2f_c$: attenuation = 0.45 (-7dB) At $f = 10f_c$: attenuation = 0.1 (-20dB)

4.3 Input Protection

STM32 GPIO absolute maximum ratings:

- Maximum voltage: VDD + 0.3V
- Minimum voltage: -0.3V (VSS - 0.3V)

Protection circuit:

Use series resistor + clamp diodes:

1. Series resistor (1-10kΩ): Limits current during overvoltage
2. Schottky diodes to VDD and GND: Clamp voltage to safe range

Current limiting calculation:

If input could reach 12V with 1kΩ series resistor:

$$I_{max} = \frac{12V - 3.6V}{1k\Omega} = 8.4mA$$

This is within the diode's current rating and won't damage the MCU.

5 Digital Filtering

5.1 Moving Average Filter

Algorithm:

$$y[n] = \frac{1}{N} \sum_{i=0}^{N-1} x[n-i]$$

Efficient implementation using circular buffer:

```c
#define FILTER_SIZE 8

uint16_t buffer[FILTER_SIZE] = {0};
uint8_t index = 0;
uint32_t sum = 0;

uint16_t moving_average(uint16_t sample) {
    sum -= buffer[index];      // Remove oldest
    buffer[index] = sample;    // Store new
    sum += sample;             // Add new
    index = (index + 1) % FILTER_SIZE;
    return sum / FILTER_SIZE;
}
```

Frequency response:

The moving average filter is a low-pass filter with:

- First null at f_s/N (where f_s is sample rate)
- -3dB cutoff approximately at $0.44 \times f_s/N$

Step response:

Linear ramp from old value to new value over N samples.

5.2 Exponential Moving Average (EMA)

Algorithm:

$$y[n] = \alpha \cdot x[n] + (1 - \alpha) \cdot y[n-1]$$

Implementation:

```c
float ema_filter(float sample, float alpha) {
    static float output = 0;
    static int initialized = 0;

    if (!initialized) {
        output = sample;
        initialized = 1;
    }

    output = alpha * sample + (1.0f - alpha) * output;
```

```
    return output;
}
```

Characteristics:

Alpha	Equivalent MA size	Response	Smoothing
0.5	~2 samples	Fast	Low
0.25	~4 samples	Medium	Medium
0.125	~8 samples	Slow	High
0.0625	~16 samples	Very slow	Very high

Relationship: $\alpha \approx 2/(N+1)$ where N is equivalent MA length.

5.3 Choosing Filter Parameters

Factors to consider:

1. **Signal bandwidth:** How fast does the measured quantity change?
2. **Noise characteristics:** Random vs periodic noise
3. **Response time requirement:** How quickly must you respond?
4. **Memory constraints:** MA needs buffer, EMA needs single variable

Guidelines:

Application	Recommended Filter	Parameters
Temperature	EMA or MA-16	$= 0.1$ or $N = 16$
Light level	MA-8	$N = 8$
Battery voltage	EMA	$= 0.05$
User input (pot)	MA-4 or none	$N = 4$

6 Calibration

6.1 Why Calibrate?

Sources of error:

1. **Reference voltage error:** VDDA may not be exactly 3.3V
2. **Sensor offset:** Zero-point error
3. **Sensor gain error:** Sensitivity variation

4. **Non-linearity:** Deviation from ideal straight line
5. **ADC errors:** Offset, gain, INL, DNL

6.2 Two-Point Calibration

Procedure:

1. Apply known input #1 (e.g., ice water at 0°C)
2. Record ADC reading: ADC_1
3. Apply known input #2 (e.g., boiling water at 100°C)
4. Record ADC reading: ADC_2
5. Calculate calibration constants

Calibration equations:

$$slope = \frac{True_2 - True_1}{ADC_2 - ADC_1}$$

$$offset = True_1 - slope \times ADC_1$$

$$Output = slope \times ADC_{raw} + offset$$

Implementation:

```c
typedef struct {
    float slope;
    float offset;
} CalibrationData_t;

CalibrationData_t temp_cal = {0};

void calibrate(uint16_t adc_low, float true_low,
               uint16_t adc_high, float true_high) {
    temp_cal.slope = (true_high - true_low) / (adc_high - adc_low);
    temp_cal.offset = true_low - temp_cal.slope * adc_low;
}

float apply_calibration(uint16_t adc_raw) {
    return temp_cal.slope * adc_raw + temp_cal.offset;
}
```

6.3 Storing Calibration Data

Options:

1. **Hardcoded constants:** Simple but requires recompilation
2. **Flash memory:** Persistent, survives power cycles
3. **EEPROM/external memory:** Easy to update in field

Flash storage example:

```
// Store at fixed flash address (last page)
#define CAL_FLASH_ADDR  0x0803F800

typedef struct {
    uint32_t magic;      // Validation marker
    float temp_slope;
    float temp_offset;
    float light_slope;
    float light_offset;
    uint32_t checksum;
} StoredCalibration_t;
```

7 Practical Considerations

7.1 ADC Noise Reduction

Hardware techniques:

1. Use separate analog ground (VSSA) connected at single point
2. Add 100nF + 1µF capacitors on VDDA
3. Keep analog traces short and away from digital signals
4. Use ground plane under analog circuits

Software techniques:

1. Oversampling and averaging
2. Discard first few samples after channel switch
3. Sample during quiet periods (no PWM transitions)

7.2 Multi-Channel Considerations

When scanning multiple channels:

1. Allow settling time between channels
2. Consider channel-to-channel crosstalk
3. Use consistent sampling order for timing analysis

Conversion sequence:

```
// Read temperature (channel 0)
select_channel(0);
delay_us(1);  // Settling time
temp_raw = read_adc();

// Read light (channel 1)
select_channel(1);
delay_us(1);
light_raw = read_adc();
```

7.3 Error Handling

Fault detection:

```
#define ADC_MIN_VALID   50    // ~40mV
#define ADC_MAX_VALID   4000  // ~3.2V

typedef enum {
    SENSOR_OK,
    SENSOR_OPEN,      // Wire disconnected
    SENSOR_SHORT,     // Shorted to GND or VCC
    SENSOR_NOISY      // Excessive noise
} SensorStatus_t;

SensorStatus_t check_sensor(uint16_t raw, uint16_t filtered) {
    if (raw < ADC_MIN_VALID)
        return SENSOR_SHORT;  // Likely shorted to GND

    if (raw > ADC_MAX_VALID)
        return SENSOR_OPEN;   // Likely disconnected

    if (abs(raw - filtered) > 100)
```

```
        return SENSOR_NOISY;  // Excessive noise

    return SENSOR_OK;
}
```

8 Quick Reference

8.1 Key Formulas

Formula	Description
$V = ADC \times V_{ref}/4095$	ADC to voltage
$V_{div} = V_{CC} \times R_2/(R_1 + R_2)$	Voltage divider
$f_c = 1/(2\pi RC)$	RC filter cutoff
$y = \alpha x + (1 - \alpha)y_{prev}$	EMA filter

8.2 HAL Functions

Function	Purpose
`HAL_ADC_Start()`	Start conversion
`HAL_ADC_PollForConversion()`	Wait for completion
`HAL_ADC_GetValue()`	Read result
`HAL_ADC_Start_DMA()`	Start DMA transfer

8.3 Common Mistakes

Mistake	Consequence	Fix
Wrong sampling time	Incorrect readings	Increase for high-Z sources
No anti-alias filter	Aliased noise	Add RC filter
Integer overflow in averaging	Wrong result	Use uint32_t for sum
Forgetting to enable ADC clock	ADC doesn't work	Check RCC configuration

9 Summary

This week covered the essential concepts for interfacing analog sensors with STM32 microcontrollers:

1. **Sensor types:** Resistive sensors need voltage dividers; voltage output sensors connect directly to ADC
2. **ADC configuration:** Resolution, sampling time, and reference voltage all affect accuracy
3. **Signal conditioning:** Voltage dividers, filters, and protection circuits prepare signals for ADC
4. **Digital filtering:** Moving average and EMA reduce noise at the cost of response time
5. **Calibration:** Two-point calibration corrects offset and gain errors

The lab will apply these concepts to build a working temperature and light sensing system with proper filtering and calibration.